Pizza42 + Auth0 Interview Deep Dive

This guide explains the Pizza42 Auth0 challenge in detail, including OAuth 2.0, OIDC, token flows, backend security, Auth0 architecture decisions, and likely interview questions. The goal is to understand not just WHAT to build, but WHY each design decision matters.

1. What the Interview is Really Testing

The Pizza42 challenge is not just a coding assignment. The panel is evaluating: • Technical understanding of OAuth 2.0, OIDC, Auth0, APIs, and JWTs • Communication and customer-facing presentation skills • Ability to explain tradeoffs and architecture decisions • Ability to connect technical features to business goals This is a pre-sales specialist role. They care about how well you explain technology to both technical and non-technical stakeholders.

---

2. Why We Chose a SPA

Pizza42 uses a React SPA (Single Page Application). A SPA runs entirely in the browser, which makes it a public client. Public clients cannot safely store secrets. That is why the correct modern flow is: OIDC + Authorization Code Flow + PKCE + Universal Login NOT the old Implicit Flow.

---

3. Universal Login

Universal Login means Auth0 hosts the authentication experience. Benefits: • Reduced security liability • Social login support • Passwordless/passkeys support • MFA support • Faster rollout • Centralized authentication Universal Login handles AUTHENTICATION. OAuth access tokens handle AUTHORIZATION.

---

4. OIDC vs OAuth 2.0

OAuth 2.0 answers: 'What can this application do?' OIDC answers: 'Who is the user?' OIDC is built on top of OAuth 2.0. In Pizza42: • OIDC handles login and identity • OAuth handles backend API authorization

---

5. Why We Created the Pizza42 API in Auth0

The Pizza42 API registration is NOT the backend server itself. It is simply an Auth0 configuration entry that defines: • Audience identifier • Available scopes • Signing algorithm Example: https://pizza42-api This allows Auth0 to issue secure access tokens specifically intended for the Pizza42 backend API.

---

6. Understanding the Tokens

ID Token: • Used by the SPA • Represents user identity • Contains claims like name, email, picture Access Token: • Used by the backend API • Represents permissions/authorization • Backend validates this before allowing orders Refresh Token: • Used to obtain new access tokens • Usually not necessary for a small PoC

---

7. Why the Backend Trusts the Access Token

The backend should NOT trust frontend values directly. The backend validates: • Signature • Issuer • Audience • Expiration • Scope/permissions This prevents attackers from bypassing frontend restrictions.

---

8. Why PKCE Matters

PKCE protects the Authorization Code Flow for SPAs. The SPA creates: • Code verifier • Code challenge Even if an attacker steals the authorization code, they cannot exchange it without the original verifier. PKCE replaced the Implicit Flow because it is much more secure.

---

9. Why We Need a Backend API

The backend API securely processes pizza orders. The SPA sends the access token to the backend. The backend: • Validates the JWT • Checks scopes • Checks email verification • Creates the order The frontend is NOT the security boundary. The backend API is the real enforcement point.

---

10. Why M2M Exists

The backend uses a Machine-to-Machine (M2M) application to securely call the Auth0 Management API. The SPA cannot safely hold admin credentials because browser apps are public. The M2M app: • Uses Client Credentials Flow • Holds secrets securely on the server • Updates metadata • Reads user records

---

11. user_metadata vs app_metadata

user_metadata: • User-owned data • Preferences • Potentially editable by the user app_metadata: • System-owned data • Trusted business data • Editable only by backend/admin systems For production architecture, order history belongs in app_metadata or preferably a real database.

---

12. Email Verification Strategy

Pizza42 allows unverified users to log in but blocks them from placing orders. Backend flow: 1. Validate access token 2. Check create:orders scope 3. Fetch current user record from Auth0 Management API 4. Check email_verified 5. Allow or deny order This design keeps verification status fresh in real time.

---

13. Passkeys

Passkeys directly solve the business problems in the brief. Benefits: • No password memorization • Reduced password reset support load • Phishing-resistant authentication • Better customer UX Passkeys improve BOTH: • Security • User experience That is why they are such a strong recommendation for Pizza42.

---

14. RBAC vs Scopes

Scopes are lightweight permissions: Example: create:orders RBAC introduces roles: • customer • manager • admin For the PoC, scopes are sufficient because all customers have the same capability.

---

15. Account Linking

By default, Auth0 treats identities from different providers separately. Example: • Email/password account • Google account with same email Without account linking, these become two separate identities.

---

16. Production vs PoC

Production improvements would include: • Real database • Environment separation • Secrets management • Monitoring/log streams • Custom domains • CI/CD for tenant configuration • Fraud detection • MFA/adaptive authentication This demonstrates understanding of enterprise deployment practices.

---

17. How to Stand Out

The strongest candidates: • Explain business value, not just code • Tie features to stakeholder pain points • Explain tradeoffs clearly • Stay calm under questioning • Think like a trusted advisor A strong pattern is: Business Problem → Technical Solution → Business Outcome

---

Final Thought: The Pizza42 challenge is ultimately about demonstrating that you understand modern customer identity architecture and can communicate it clearly to both technical and business audiences. The build matters. But the explanation matters even more.